1)

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

#define MAX_SIZE 100

typedef struct Stack {
   int top;
   double arr[MAX_SIZE];
} Stack;

void initStack(Stack *s) {
   s->top = -1;
}

int isFull(Stack *s) {
   return s->top == MAX_SIZE - 1;
}

int isEmpty(Stack *s) {
   return s->top == -1;
}

void push(Stack *s, double val) {
   if(isFull(s)) {
      printf("Stack overflow\n");
      return;
   }
   s->arr[++(s->top)] = val;
}

double pop(Stack *s) {
   if(isEmpty(s)) {
      printf("Stack underflow\n");
      return -1;
   }
   return s->arr[(s->top)--];
}

double evaluatePrefix(char *expr) {
   Stack stack;
   initStack(&stack);

   int length = strlen(expr);
```

```
    for(int i = length - 1; i >= 0; i--) {
        if(isdigit(expr[i])) {
            push(&stack, (double)(expr[i] - '0'));
        } else {
            double operand1 = pop(&stack);
            double operand2 = pop(&stack);

            switch(expr[i]) {
                case '+':
                    push(&stack, operand1 + operand2);
                    break;
                case '-':
                    push(&stack, operand1 - operand2);
                    break;
                case '*':
                    push(&stack, operand1 * operand2);
                    break;
                case '/':
                    if(operand2 == 0.0) {
                        printf("Error: Division by zero!\n");
                        return -1;
                    }
                    push(&stack, operand1 / operand2);
                    break;
            }
        }
    }
    return pop(&stack);
}

int main() {
    char expr[MAX_SIZE];
    printf("Enter the prefix expression: ");
    scanf("%s", expr);

    double result = evaluatePrefix(expr);
    printf("Result: %.2lf\n", result);
    return 0;
}

Enter the prefix expression: *+823
Result: 30.00
```

2)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_SIZE 100

typedef struct Stack {
   int top;
   char arr[MAX_SIZE];
} Stack;

void initStack(Stack *s) {
   s->top = -1;
}

int isFull(Stack *s) {
   return s->top == MAX_SIZE - 1;
}

int isEmpty(Stack *s) {
   return s->top == -1;
}

void push(Stack *s, char val) {
   if(isFull(s)) {
      printf("Stack overflow\n");
      return;
   }
   s->arr[++(s->top)] = val;
}

char pop(Stack *s) {
   if(isEmpty(s)) {
      printf("Stack underflow\n");
      return '\0';
   }
   return s->arr[(s->top)--];
}

int precedence(char op) {
   switch(op) {
      case '+':
      case '-':
         return 1;
```

```
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
        default:
            return 0;
    }
}

void reverseStr(char* str) {
    int start = 0;
    int end = strlen(str) - 1;
    while (start < end) {
        char temp = str[start];
        str[start] = str[end];
        str[end] = temp;
        start++;
        end--;
    }
}

char* infixToPrefix(char* infix) {
    Stack s;
    initStack(&s);

    char* prefix = (char*) malloc(MAX_SIZE * sizeof(char));
    int prefixIndex = 0;

    reverseStr(infix);

    for (int i = 0; i < strlen(infix); i++) {
        if (isalnum(infix[i])) {
            prefix[prefixIndex++] = infix[i];
        } else if (infix[i] == ')') {
            push(&s, infix[i]);
        } else if (infix[i] == '(') {
            while (!isEmpty(&s) && s.arr[s.top] != ')') {
                prefix[prefixIndex++] = pop(&s);
            }
            if (!isEmpty(&s) && s.arr[s.top] == ')') {
                pop(&s);
            }
        } else {
            while (!isEmpty(&s) && precedence(infix[i]) < precedence(s.arr[s.top])) {
                prefix[prefixIndex++] = pop(&s);
            }
```

```
            push(&s, infix[i]);
        }
    }

    while (!isEmpty(&s)) {
        prefix[prefixIndex++] = pop(&s);
    }

    prefix[prefixIndex] = '\0';

    reverseStr(prefix);
    return prefix;
}

int main() {
    char infix[MAX_SIZE];
    printf("Enter infix expression: ");
    scanf("%s", infix);

    char* prefix = infixToPrefix(infix);
    printf("Prefix Expression: %s\n", prefix);

    free(prefix);
    return 0;
}
```

```
Enter infix expression: a+b*c-d
Prefix Expression: -+a*bcd
```

3)

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_SIZE 100

typedef struct {
    int arr[MAX_SIZE];
    int top1;  // For stack 1
    int top2;  // For stack 2
} TwoStacks;

void init(TwoStacks *ts) {
    ts->top1 = -1;
```

```
    ts->top2 = MAX_SIZE;
}

int isFull(TwoStacks *ts) {
    return ts->top1 + 1 == ts->top2;
}

int isEmpty1(TwoStacks *ts) {
    return ts->top1 == -1;
}

int isEmpty2(TwoStacks *ts) {
    return ts->top2 == MAX_SIZE;
}

void push1(TwoStacks *ts, int data) {
    if (isFull(ts)) {
        printf("Stack overflow\n");
        return;
    }
    ts->arr[++(ts->top1)] = data;
}

void push2(TwoStacks *ts, int data) {
    if (isFull(ts)) {
        printf("Stack overflow\n");
        return;
    }
    ts->arr[--(ts->top2)] = data;
}

int pop1(TwoStacks *ts) {
    if (isEmpty1(ts)) {
        printf("Stack 1 underflow\n");
        return -1;
    }
    return ts->arr[(ts->top1)--];
}

int pop2(TwoStacks *ts) {
    if (isEmpty2(ts)) {
        printf("Stack 2 underflow\n");
        return -1;
    }
    return ts->arr[(ts->top2)++];
}
```

```
int main() {
    TwoStacks ts;
    init(&ts);

    // Pushing to Stack 1
    push1(&ts, 1);
    push1(&ts, 2);
    push1(&ts, 3);

    // Pushing to Stack 2
    push2(&ts, 10);
    push2(&ts, 9);
    push2(&ts, 8);

    // Popping from Stack 1
    printf("Popped from Stack 1: %d\n", pop1(&ts));
    printf("Popped from Stack 1: %d\n", pop1(&ts));

    // Popping from Stack 2
    printf("Popped from Stack 2: %d\n", pop2(&ts));
    printf("Popped from Stack 2: %d\n", pop2(&ts));

    return 0;
}
```

```
Popped from Stack 1: 3
Popped from Stack 1: 2
Popped from Stack 2: 8
Popped from Stack 2: 9
```

4)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_SIZE 100

typedef struct {
    char* arr[MAX_SIZE];
    int top;
```

```
} Stack;

void initStack(Stack* s) {
   s->top = -1;
}

int isEmpty(Stack* s) {
   return s->top == -1;
}

int isFull(Stack* s) {
   return s->top == MAX_SIZE - 1;
}

void push(Stack* s, char* str) {
   if (isFull(s)) {
      printf("Stack overflow\n");
      return;
   }
   s->arr[++(s->top)] = str;
}

char* pop(Stack* s) {
   if (isEmpty(s)) {
      printf("Stack underflow\n");
      return NULL;
   }
   return s->arr[(s->top)--];
}

void reverseStr(char* str) {
   int start = 0;
   int end = strlen(str) - 1;
   while (start < end) {
      char temp = str[start];
      str[start] = str[end];
      str[end] = temp;
      start++;
      end--;
   }
}

char* prefixToPostfix(char* prefix) {
   Stack s;
   initStack(&s);

   reverseStr(prefix);
```

```
    for (int i = 0; i < strlen(prefix); i++) {
        char* temp = (char*) malloc((strlen(prefix)+1) * sizeof(char));

        if (isalnum(prefix[i])) {
            snprintf(temp, strlen(prefix)+1, "%c", prefix[i]);
            push(&s, temp);
        } else {
            char* operand1 = pop(&s);
            char* operand2 = pop(&s);

            snprintf(temp, strlen(prefix)+1, "%s%s%c", operand1, operand2, prefix[i]);
            push(&s, temp);

            free(operand1);
            free(operand2);
        }
    }
    return pop(&s);
}

int main() {
    char prefix[MAX_SIZE];
    printf("Enter prefix expression: ");
    scanf("%s", prefix);

    char* postfix = prefixToPostfix(prefix);
    printf("Postfix Expression: %s\n", postfix);

    free(postfix);
    return 0;
}
```

```
Enter prefix expression: *+ABC
Postfix Expression: AB+C*
```